

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Abstract Methods & Classes

Data Abstraction

□ Data **abstraction** is the process of hiding certain details and showing only essential information to the user.

□ Abstraction can be achieved with either **abstract classes** or **interfaces**.

The **abstract** keyword is a non-access modifier, used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

Abstract Class

An **abstract class** in Java is a class that **cannot be instantiated** on its own. It may contain:

- Abstract methods (methods without a body)
- Concrete methods (with full implementation)

You use it when you want a **base class that only serves as a blueprint** for other classes.

Abstract Class

An abstract class can have both abstract and regular methods:

```
abstract class Animal {  
    public abstract void animalSound();  
    public void sleep() {  
        System.out.println("Zzz");  
    }  
}
```

From the example above, it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // will generate an error
```

Abstract class (Cont...)

Let's consider a class Animal

```
class Animal {  
    void makeSound() {  
        System.out.println("Animals make sounds");  
    }  
}
```

Currently, this class has a default implementation of makeSound().

But you don't want Animal to be directly instantiated because it doesn't make sense to have a generic animal without specifying what kind it is.

You want every subclass (e.g., Dog, Cat, Lion) to define its own sound. Then you have to make Animal an abstract class with an abstract method.



2



3



4

You enforce a contract: all subclasses must implement the abstract methods.



You prevent generic instantiation: e.g., no one can create just a plain Animal.



You allow code reuse via common methods or fields that apply to all subclasses.

Why use
abstract
class?

Abstract method

An *abstract method* is a method that is declared without an implementation (without braces, and followed by a semicolon), like this:

```
abstract type name(parameter-list);
```

- If a **class includes abstract methods**, then the **class itself *must* be declared abstract**.
- When an **abstract method appears in a class**, the **method must be overridden in a subclass**. If a subclass fails to override the method, an error will result.
- Abstract methods are used to ensure that a subclass implements the method.

// Abstract class

abstract class Animal {

// Abstract method (no body)

public abstract void animalSound();

// Regular method

public void sleep() {

 System.out.println("Zzz");

}

}

// Subclass that overrides the abstract method

class Dog extends Animal {

@Override

public void animalSound() {

 System.out.println("The dog says: woof
woof");

}

}

// Main class to test the functionality

public class Main {

public static void main(String[] args) {

 Dog myDog = new Dog(); // Create a Dog object

 myDog.animalSound(); // Calls the overridden

method

 myDog.sleep(); // Calls the inherited method

}

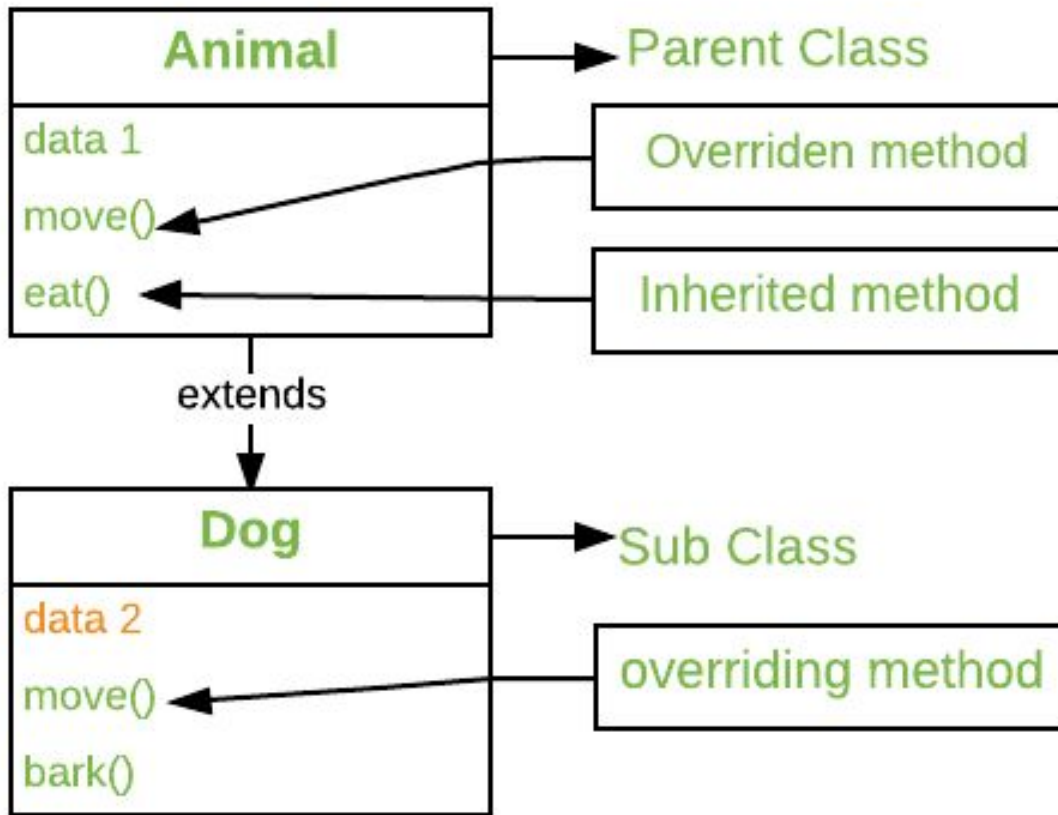
}

The dog says: woof woof
Zzz

Method Overriding

Method overriding occurs when a subclass provides its own specific implementation of a method that is already defined in its parent class.

The overridden method in the subclass must have the same name, return type, and parameters as the method in the parent class.



To customize or extend behavior inherited from a parent class. For example:

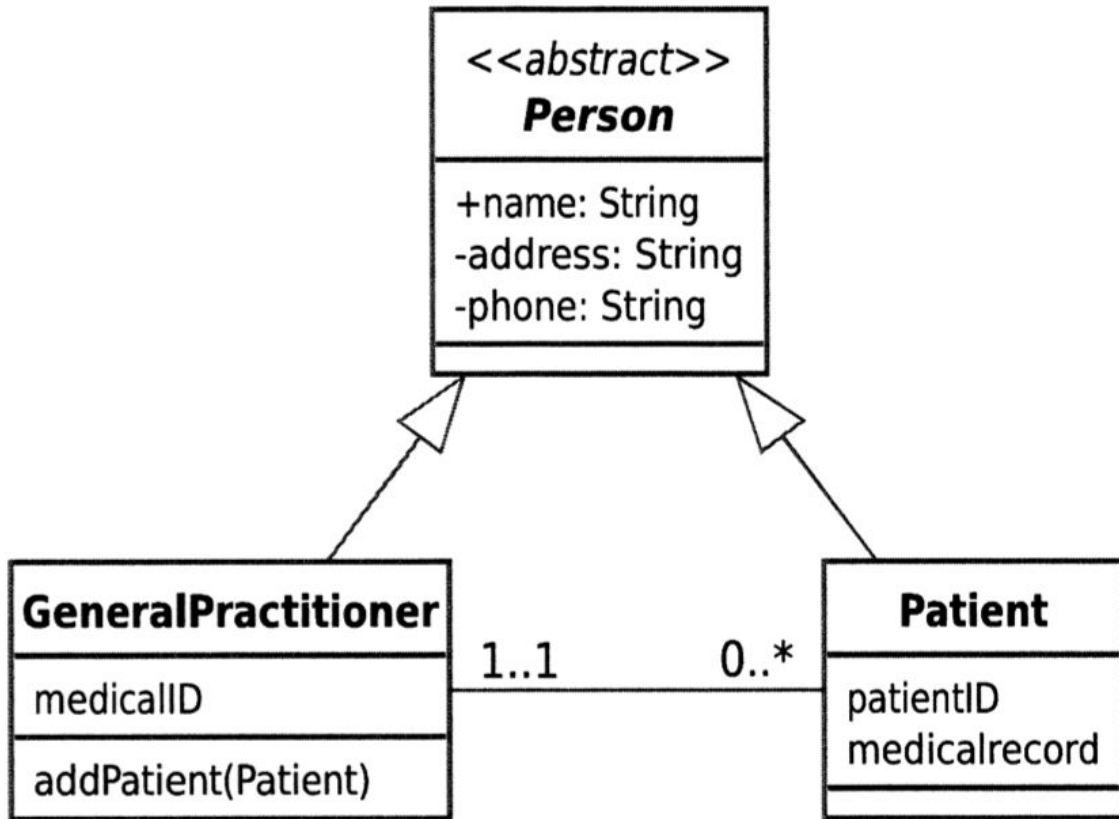
- A Bird might override a **move()** method to say "flies"
- A Fish might override **move()** to say "swims"

```
// Parent class
class Animal {
    void makeSound() {
        System.out.println("Animals make
sounds");
    }
}
```

```
// Subclass overriding the makeSound
method
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("The dog says: Woof
woof!");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Creating an Animal object
        Animal myAnimal = new Animal();
        myAnimal.makeSound(); // Output:
Animals make sounds

        // Creating a Dog object using Animal
reference (polymorphism)
        Animal myDog = new Dog();
        myDog.makeSound(); // Output: The dog
says: Woof woof!
    }
}
```



Representation
of Abstract Class
in UML Class
Diagram

The final Keyword

In Java, the final is a reserved keyword. It is use to restrict the user.

It is used for following three purposes.

1. To declare a variable as constant.
2. To prevent overriding
3. To prevent inheritance.

To Declare A Variable As Constant

You can assign a value to final variable only once and that value can't be changed throughout the program.

In java final variable is treated same as constant variable.

The syntax is given below.

final datatype var_name= value;

```
class Bike9{  
    final int speedlimit=90;//final variable  
  
    void run(){  
        speedlimit=400;  
    }  
    public static void main(String args[]){  
  
        Bike9 obj=new Bike9();  
        obj.run();  
    }  
} //end of class
```

To Prevent Overriding

Sometimes to prevent a method to be overridden by its subclasses we can use final key word to declare a given method.

Syntax

```
final returntype method_name(parameters list)  
{  
//body of the method;  
}
```

Using final with inheritance

To prevent overriding

- final method

```
class x
{
    void show()
    {
        int i=20;
        System.out.println(" hello");
    }
}
class y extends x
{
    void show()
    {
        System.out.println(" Hi");}
}
```

```
public class Final_k {
    public static void main(String args[])
    {
        y obj=new y();
        obj.show();
    }
}
```

```
class x
{
    final void show()
    {
        System.out.println(" hello");
    }
}
class y extends x
{
    void show1()
    {
        System.out.println(" Hi");}
}
```

```
public class Final_k {
    public static void main(String args[])
    {
        y obj=new y();
        obj.show();
        obj.show1();
    }
}
```


To Prevent Inheritance

Sometimes we need to prevent the class to be inherited by any other class.

In such case **you need to declare the class using final keyword.**

When the class is declared as final the variables and keywords in it are also declared as final.

Syntax

```
final class classname  
{  
//body of the class;  
}
```

Using final with inheritance

To prevent inheritance

- final class

```
final class Bike{}
```

```
class Honda1 extends Bike{  
    void run(){System.out.println("running safely  
with 100kmph");}
```

```
    public static void main(String args[]){  
        Honda1 honda= new Honda1();  
        honda.run();  
    }  
}
```

```

import java.lang.System;
class tes
{
    int x;
    tes(int n)
    {
        this.x=n;
    }
    public String toString()
    {
        return "(" + x + ")";
    }
}
public class test_string
{
    public static void main(String args[])
    {
        String name="java";
        String x;
        x=new String("hello");
        char arr[]={'A','M','M','A','R','A'};
        String z= new String(arr,0,5); //array name, starting index , total char
        System.out.println(name);
        System.out.println(x);
        System.out.println(z);
        tes t=new tes(10);
        System.out.println(t);}}

```

Find Output?

Java Interface

Interface in Java

An interface in Java is a blueprint of a class.

It has default public, static and final fields.

It has default public, abstract methods.

It is used to achieve abstraction and multiple inheritance in Java.

Constructors are not allowed in interfaces

You cannot create objects of interfaces.

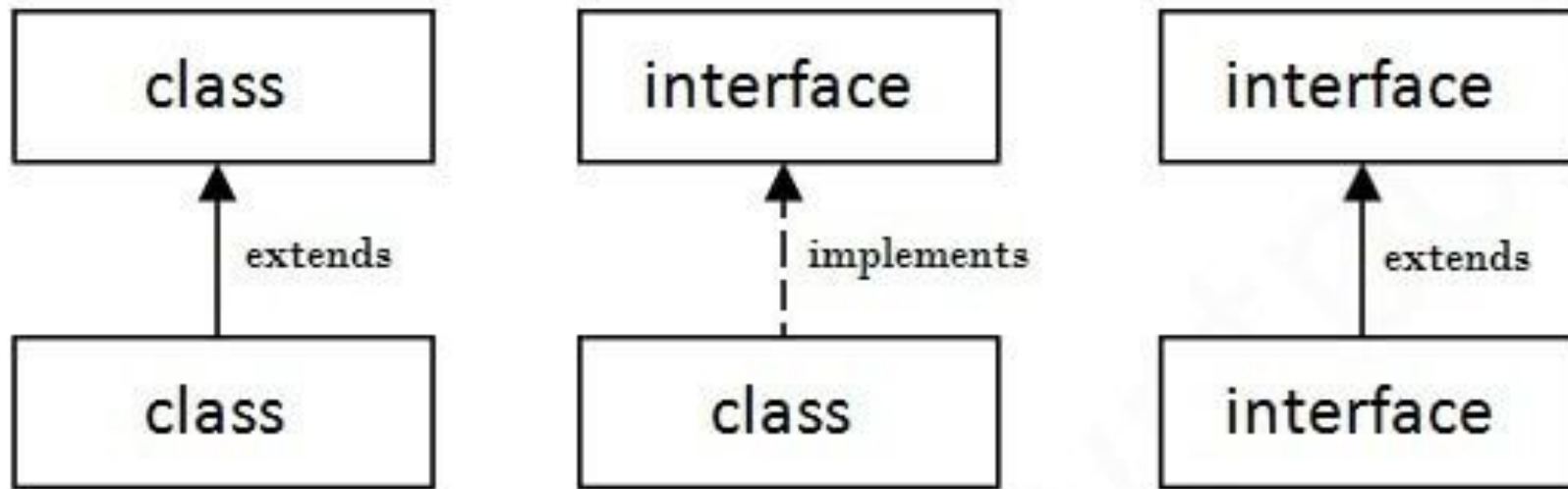
Declare an interface

An interface is declared by using the interface keyword.

A class that implements an interface must implement all the methods declared in the interface.

```
interface interface_name{  
  
    // declare constant fields  
    // declare methods that abstract  
    // by default.  
}
```

Relation b/w Classes and Interface



A class extends another class, an interface extends another interface, but a **class implements an interface**.

Java Interface

An interface is a completely "**abstract class**" that is used to group related methods with empty bodies:

Example

```
// interface
interface Animal {
    public void animalSound(); // interface method (does not have a body)
    public void run(); // interface method (does not have a body)
}
```


Con..

To access the interface methods, the interface must be "implemented" (kind a like inherited) by another class with the **implements** keyword (instead of **extends**) The body of the interface method is provided by the "implement" class:

```
// Interface
interface Animal {
    void sound(); // abstract by default
}
```

// Class implementing the interface

```
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound(); // Output: Dog barks
    }
}
```

Implementing Interface

When you implement an interface method ,it must be declared as public

By implementing an interface , a class signs a contract with the compiler that it will definitely provide implementation of all the methods

If it fails to do so, the class will be considered as abstract

Then it must be declared as abstract and no object of that class can be created

Interface Naming

Interface names are often adjectives, ending in

- *-able or -ive.*
- examples: Colorable, Rotatable

Interface names that specify roles often end in –er

- examples: Container, Mover, Teacher

Extending Interface

An interface can extend multiple interfaces

A class can extend only one class, but implement many interfaces.

Extends interface

interface I1

```
{ void dis(); }
```

interface I2 **extends** I1

```
{ void dis2(); }
```

class test_1 implements I2

```
{ public void dis()  
    { System.out.println(" interface 1"); }  
    public void dis2()  
    { System.out.println(" interface 2"); }  
}
```

```
public class Interface_extends  
{  
    public static void main(String args[])  
    {  
        test_1 t=new test_1();  
        t.dis();  
        t.dis2();  
    }  
}
```

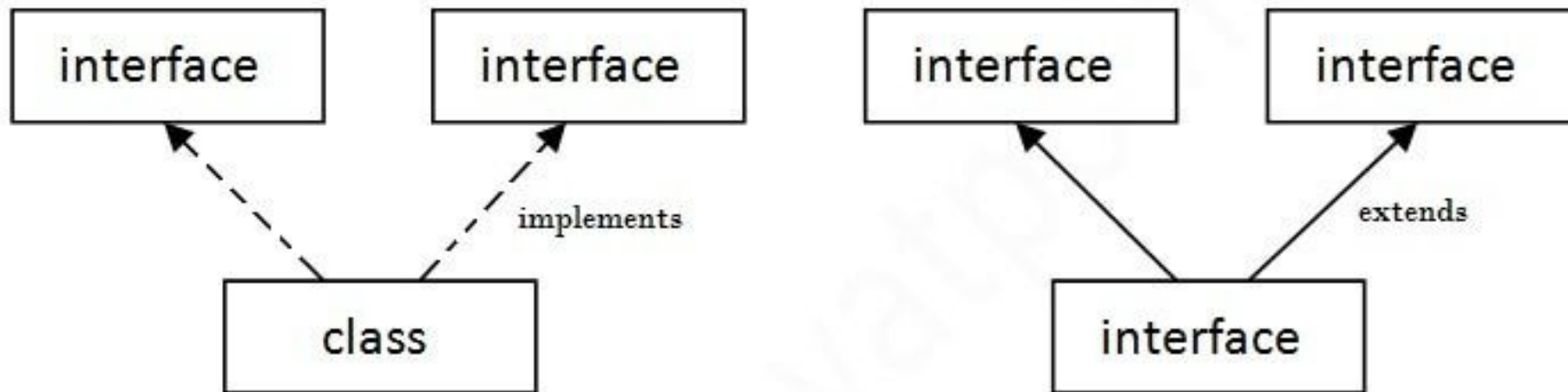
run:

```
interface 1  
interface 2
```

BUILD SUCCESSFUL (total time: 0 seconds)

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

Multiple inheritance in Java by interface

```
interface Printable{  
  void print();  
}  
interface Showable{  
  void show();  
}  
class A7 implements Printable, Showable{  
  public void print(){System.out.println("Hello...  
  Ready to Print");}  
  public void show(){System.out.println("Welco  
  me");}
```

```
public static void main(String args[]){  
  A7 obj = new A7();  
  obj.print();  
  obj.show();  
}
```

Multiple inheritance in Java by interface

```
interface Printable{  
    void print();  
}  
  
interface Showable{  
    void print();  
}
```

- Same method name and return type is acceptable in interfaces
- Same method name but **different return types** is not acceptable.

```
class TestInterface1 implements Printable, Showable  
{  
    public void print(){  
        System.out.println("Hello");  
    }  
    public static void main(String args[]){  
        TestInterface1 obj = new TestInterface1();  
        obj.print();  
    }  
}
```


Default Interface Method

Prior to JDK 8, Interfaces could only declare methods — no implementation. All methods were implicitly public abstract. If a new method was added to an interface, every implementing class would break unless it was updated.

The release of JDK 8 has changed this by adding a new capability to interface called the **default method**

- A default method lets you define a default implementation for an interface method
- Solves the problem of breaking backward compatibility when you evolve interfaces.

Default Interface Methods

```
interface Drawable{  
    void draw();  
    default void msg(){System.out.println("default method");}  
}  
class Rectangle implements Drawable{  
    public void draw(){System.out.println("drawing rectangle");}  
  
}  
class TestInterfaceDefault{  
    public static void main(String args[]){  
        Rectangle d=new Rectangle();  
        d.draw();  
        d.msg();  
    }  
}
```

```
interface Vehicle {
    default void start() {
        System.out.println("Starting the vehicle...");
    }

    void stop(); // still abstract
}

class Car implements Vehicle {
    public void stop() {
        System.out.println("Stopping the car.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.start(); // uses default method from interface
        myCar.stop(); // implemented by class
    }
}
```

Interface vs Abstract Class

- ❑ A class can implement many interfaces but can only have one superclass
- ❑ An interface is not part of the class hierarchy. Unrelated classes can implement the same interface.
- ❑ Use abstract class when you want a template for a collection of subclasses
- ❑ Subclass when you want to extend a class and add some functionality , whether or not the parent class is abstract
- ❑ Define an interface when there is no common parent class with desired functionality , and when you want only certain unrelated classes to have that functionality

Notes on Interfaces

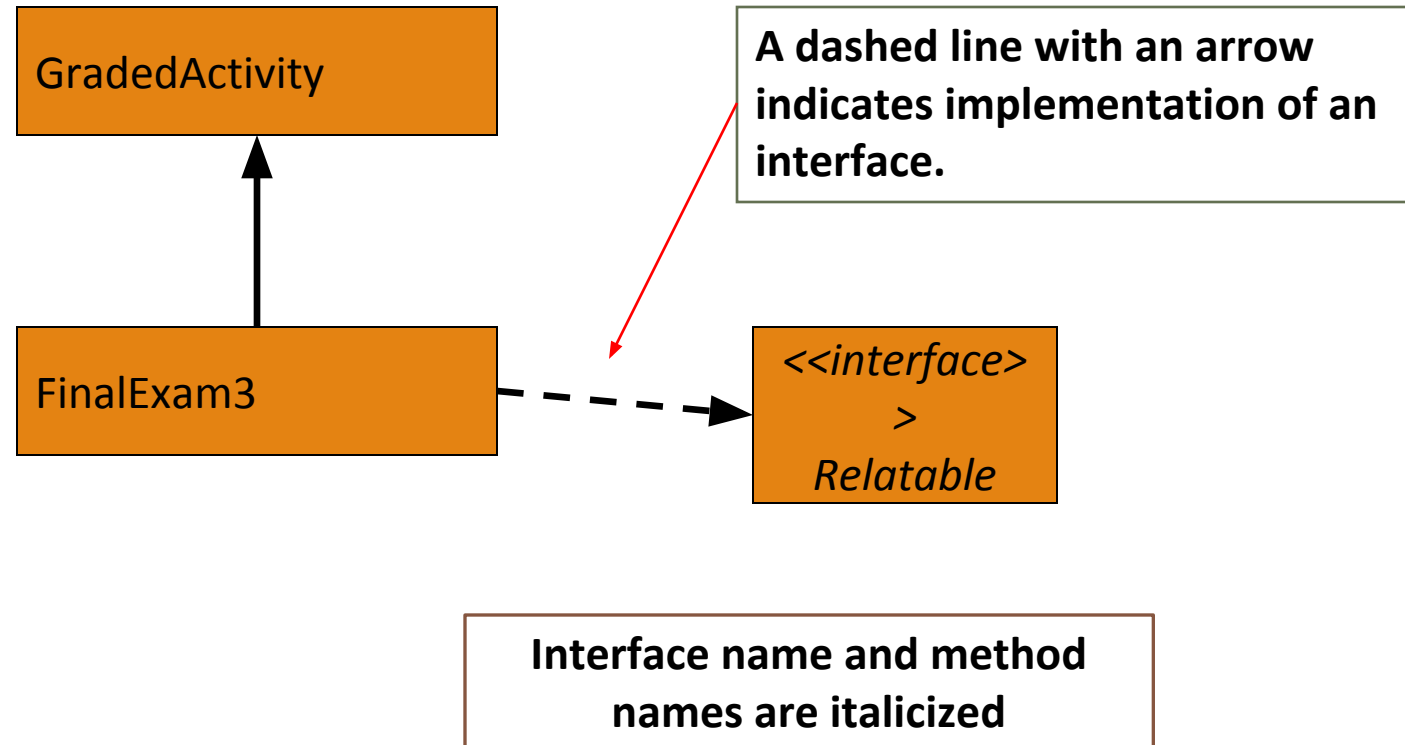
- ❑ Like **abstract classes**, interfaces **cannot** be used to create objects (in the example above, it is not possible to create an "Animal" object in the MyMainClass)
- ❑ Interface methods do not have a body - the body is provided by the "implement" class
- ❑ On implementation of an interface, you must override all of its methods
- ❑ Interface methods are by default **abstract** and **public**
- ❑ Interface attributes are by default **private** , **static** and **final**
- ❑ An interface cannot contain a constructor (as it cannot be used to create objects)

Why And When To Use Interfaces?

1. To achieve security - hide certain details and only show the important details of an object (interface).
2. Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can **implement** multiple interfaces.

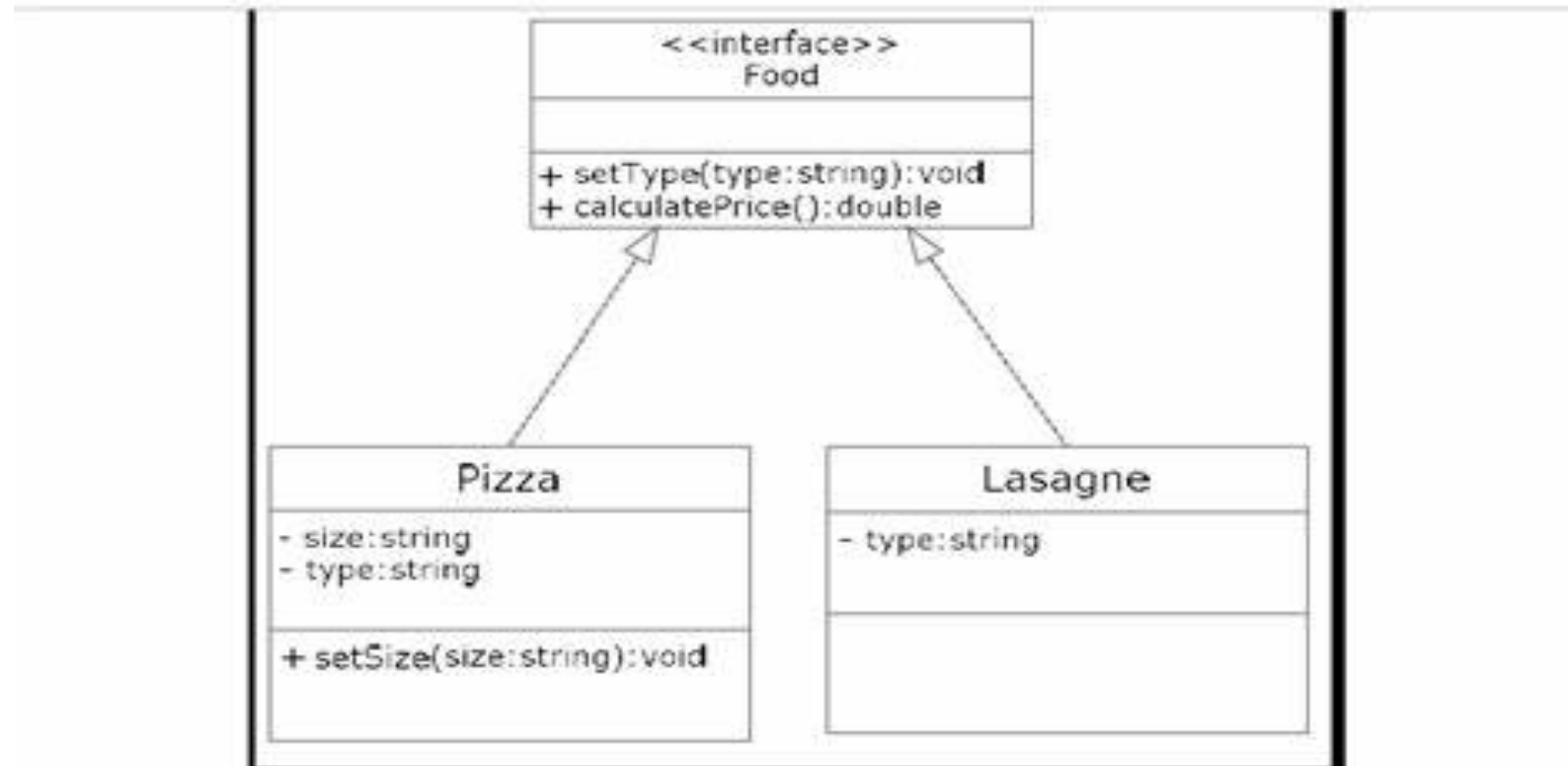
Note: To implement multiple interfaces, separate them with a comma.

Interfaces in UML



Task

Implement the multiple inheritance using the following UML Class Diagram.



Class Diagram Example: Order System

